```
import sys
import os
import logging
import camelot

logging.basicConfig(level=logging.INFO, format="%(message)s")
log = logging.getLogger("camelot-demo")

BACKENDS = ["pdfium", "ghostscript", "poppler"]
FLAVORS = ["stream", "network", "hybrid", "auto"]


def make_sample_pdf(path):
    from reportlab.lib.pagesizes import letter
    from reportlab.platypus import SimpleDocTemplate, Table, TableStyle, PageBrea
    from reportlab.lib import colors

    header = ["ID", "Name", "Department", "Salary"]
    rows_page1 = [header] + [[str(i), f"Employee {i}", "Engineering", f"${50000 +
    rows_page2 = [header] + [[str(i), f"Employee {i}", "Engineering", f"${50000 +

    doc = SimpleDocTemplate(path, pagesize=letter)
    style = TableStyle([
        ("GRID", (0, 0), (-1, -1), 1, colors.black),
        ("BACKGROUND", (0, 0), (-1, 0), colors.lightgrey),
    ])
    elements = [Table(rows_page1, style=style), PageBreak(), Table(rows_page2, st
    doc.build(elements)


def run_backend(pdf_path, backend_name, pages):
    log.info(f"\n{'=' * 60}\nBackend: {backend_name}\n{'=' * 60}")
    try:
        tables = camelot.read_pdf(
            pdf_path,
            pages=pages,
            flavor="lattice",
            backend=backend_name,
            use_fallback=False,
        )
        log.info(f"  -> found {tables.n} table(s)")
        for i, t in enumerate(tables):
            log.info(
                f"     table[{i}] page={t.page} shape={t.shape} "
```

```
                f"accuracy={t.accuracy:.1f} whitespace={t.whitespace:.1f}"
            )
        return tables
    except Exception as e:
        log.warning(f"  -> backend '{backend_name}' failed: {e}")
        return None


def run_flavors(pdf_path, pages):
    log.info(f"\n{'=' * 60}\nText-based flavors\n{'=' * 60}")
    for flavor in FLAVORS:
        try:
            flavor_tables = camelot.read_pdf(pdf_path, pages=pages, flavor=flavor
            log.info(f"flavor='{flavor}': found {flavor_tables.n} table(s)")
        except Exception as e:
            log.warning(f"flavor='{flavor}' failed: {e}")

    try:
        ml_tables = camelot.read_pdf(pdf_path, pages=pages, flavor="ml")
        log.info(f"flavor='ml': found {ml_tables.n} table(s)")
    except ImportError:
        log.info('flavor=\'ml\' skipped (install with: pip install "camelot-py[ml
    except Exception as e:
        log.warning(f"flavor='ml' failed: {e}")


def stitch_and_export(tables, out_dir):
    log.info(f"\n{'=' * 60}\nMulti-page stitching\n{'=' * 60}")
    log.info(f"Before stitching: {tables.n} physical table(s)")

    stitched_by_columns = tables.stack_contiguous(match="column_count")
    log.info(f"After stack_contiguous(match='column_count'): {stitched_by_columns

    stitched_by_header = tables.stack_contiguous(match="first_row", keep_first_he
    log.info(f"After stack_contiguous(match='first_row'):   {stitched_by_header.n

    for i, t in enumerate(stitched_by_columns):
        log.info(f"\n--- Stitched table {i} (page={t.page}, shape={t.shape}) ---"
        print(t.df.head(10).to_string())

    os.makedirs(out_dir, exist_ok=True)
    for i, t in enumerate(stitched_by_columns):
        out_csv = os.path.join(out_dir, f"stitched_table_{i}.csv")
        t.to_csv(out_csv)
        log.info(f"Wrote {out_csv}")

    return stitched_by_columns
```

```
def main():
    pdf_path = sys.argv[1] if len(sys.argv) > 1 else "sample.pdf"
    pages = "all"
    out_dir = "camelot_output"

    if not os.path.exists(pdf_path):
        log.info(f"No PDF found at '{pdf_path}', generating a sample multi-page t
        make_sample_pdf(pdf_path)

    results = {}
    for backend in BACKENDS:
        results[backend] = run_backend(pdf_path, backend, pages)

    tables = results.get("pdfium") or results.get("poppler") or results.get("ghos
    if tables is None or tables.n == 0:
        log.error("No backend produced tables — check pdf_path and installed depe
        sys.exit(1)

    run_flavors(pdf_path, pages)
    stitch_and_export(tables, out_dir)


if __name__ == "__main__":
    main()
```